LangChain-M-Set1

FDE / Architect practice | Reasoning, alternatives, and design takeaways

01. Input shape at a passthrough boundary

Answer C: Replace the question branch with itemgetter("question").

RunnablePassthrough returns the entire input unchanged. Here that input is a dictionary, so the question branch produces a nested dictionary rather than the requested string. Selecting question extracts the correct field while leaving retrieval intact. The defect is a data-shape mismatch at composition time, not model behavior.

Why the alternatives fail

A. Parsing document output does not change what the question branch receives or returns.

B. The whole mapping also creates context; bypassing it removes the retrieval result expected by the prompt.

D. The context branch still uses itemgetter("question"), which requires the dictionary-shaped input.

Design takeaway: Trace the exact input and output type at every LCEL boundary.

Official references: [1]

02. Sequential enrichment with assign

Answer A: Use one assign for tax, followed by a second assign for total.

The functions within one assign mapping receive the input to that assignment; keyword order does not establish a dependency. Because tax is absent from the original dictionary, total cannot read it there. Two assignments create the required sequence: the first adds tax=10, and the second receives that enriched dictionary and computes total=110.

Why the alternatives fail

B. Python keyword order does not turn sibling computations into a pipeline.

C. Concurrency changes scheduling capacity, not which fields exist in the input.

D. A parallel mapping has the same missing-input dependency unless the first calculation is performed upstream.

Design takeaway: Use separate enrichment stages when one derived field depends on another.

Official references: [2]

03. Reading LCEL data flow

Answer D: {"a": 7, "b": 12}; both branches receive 6.

The first runnable doubles 3 to produce 6. The dictionary in the sequence is converted to a RunnableParallel, whose branches both consume that 6. Branch a adds 1 and returns 7; branch b multiplies by 2 and returns 12. The output is a mapping, not the final result of a sequential chain of branches.

Why the alternatives fail

A. A dictionary composition collects named branch outputs rather than flattening them into one value.

B. There is no edge from a's output to b's input in this expression.

C. Both branches receive the preceding runnable's output, not the original chain input.

Design takeaway: The pipe passes outputs forward; a mapping distributes the same input to its branches.

Official references: [3]

04. A hidden streaming barrier

Answer B: The full-input transform buffers upstream output; use a chunk-aware transform if normalization permits it.

A transform that consumes one complete string cannot emit a normalized final result before that string is available. A normal RunnableLambda can introduce this barrier. If the operation is valid incrementally, implement a streaming-aware generator or transform. If it requires global context, retain buffering and make that latency trade-off explicit.

Why the alternatives fail

A. The timing change coincides with the appended full-input stage; the provider is not the first demonstrated cause.

C. More workers do not change a function's requirement for complete input.

D. invoke collects a final output; it does not turn a whole-string function into a streaming transform.

Design takeaway: End-to-end streaming depends on every stage, not just the model.

Official references: [4]

05. Associating completion-order results

Answer A: Use the returned input index to attach each result to its original document.

batch_as_completed returns an input index with each output. Completion order reflects execution time, so position in the stream is not identity. Use the returned index to look up the original document or its stable business ID. This also lets the consumer record an individual failure against the correct input when exceptions are returned.

Why the alternatives fail

B. Zipping assumes arrival order equals input order, which the scenario explicitly disproves.

C. Label text has no reliable relationship to source-document identity.

D. Arrival position describes scheduling, not the index supplied by the API.

Design takeaway: Preserve explicit input identity when consuming completion-order results.

Official references: [5]

06. Tool proposals versus execution

Answer C: A tool execution loop, such as create_agent or an explicitly implemented dispatcher.

Binding tools makes their schemas available to the model; it does not run the proposed functions by itself. A surrounding agent loop or dispatcher validates and executes the call, supplies the result, and continues model interaction. Execution permission must be enforced at that boundary rather than inferred from the model choosing the tool.

Why the alternatives fail

A. Sampling settings influence model output, not whether application code invokes a function.

B. Repeating schema binding does not add a runtime dispatcher.

D. A string parser extracts text; interpreting text as executable code is not a tool execution contract.

Design takeaway: Tool calling produces a request; the application or agent harness executes it.

Official references: [6]

07. Validation does not establish evidence

Answer D: Check extracted values, units, and currency against supporting source spans.

The values can satisfy the declared types while misrepresenting the source. Validation guarantees only the constraints actually encoded. Add evidence-based checks for currency, scale, period, and numerical consistency; preserve supporting spans so failures can be inspected. Custom validators can enforce internal rules, but external factual correctness still requires evidence.

Why the alternatives fail

A. Parsing the same values again cannot establish their factual relationship to the document.

B. A more permissive type does not correct a wrong unit or currency.

C. The schema did not encode or verify the source-specific facts.

Design takeaway: Separate structural validity, business-rule validity, and source-grounded accuracy.

Official references: [7]

08. Selecting a structured-output mechanism

Answer B: Use ToolStrategy(schema) and handle structured-output validation failures.

ToolStrategy uses a tool-calling mechanism to obtain structured output for models that support tools. The resulting response still needs schema validation and a bounded error-handling policy. Provider-native structured output is a different capability; the existence of ordinary tool calling does not imply native schema enforcement.

Why the alternatives fail

A. A configuration cannot create a native capability absent from the selected provider/model.

C. Disabling the structured response contract does not produce a validated object automatically.

D. A description may guide generation, but it does not enforce the output contract.

Design takeaway: Match the structured-output strategy to verified model capabilities.

Official references: [8]

09. Preserving message roles in prompts

Answer B: Replace ("human", "{history}") with MessagesPlaceholder("history").

MessagesPlaceholder inserts a list of message objects into the prompt while preserving their roles and boundaries. Formatting the whole list into one human message turns structured conversation into text. Promoting history to a system message is especially inappropriate because prior user/model content should not gain instruction authority.

Why the alternatives fail

A. This changes the authority of the content rather than preserving the original turns.

C. JSON serialization preserves a representation, not separate chat-protocol messages.

D. Merging history into the current question also collapses the role and turn structure.

Design takeaway: Use a message placeholder when the input is already structured chat history.

Official references: [9]

10. Separating retrieval intent from answer intent

Answer A: Retain the original question for generation and use rewritten queries only for retrieval.

Search-oriented rewrites optimize retrieval, not necessarily the user's full task semantics. Keep separate fields for the original question and retrieval queries. The answering stage should see the original comparison request together with the retrieved evidence. This preserves intent while allowing search transformations to improve recall.

Why the alternatives fail

B. That continues the loss of comparison intent identified in the scenario.

C. Temperature does not restore information discarded by the input contract.

D. Query variants are internal search artifacts and should not become additional user directives.

Design takeaway: Preserve the original user intent across retrieval-specific transformations.

Official references: [10]

11. Embedding-space compatibility

Answer C: Equal dimensionality does not imply aligned spaces; use a compatible query/document embedding pair.

Vector coordinates acquire meaning from the embedding model and its intended query/document encoders. Two unrelated models can emit the same number of dimensions with different geometry. Restore the compatible query encoder or rebuild and validate the index with an appropriate new pair. Some retrieval models intentionally use distinct but jointly trained query/document encoders; compatibility, not identical method names, is the requirement.

Why the alternatives fail

A. Larger k can mask some failures while increasing cost; it does not repair incompatible geometry.

B. Dimension count alone does not establish semantic alignment.

D. Padding changes representation length without aligning its semantic coordinates.

Design takeaway: Version the embedding space and validate migrations as retrieval changes.

Official references: [11]

12. Keeping citation provenance through formatting

Answer D: Preserve stable evidence IDs and a source/page mapping, then validate citations against that mapping.

Document metadata is not automatically visible to the model after a formatter discards it. Supply explicit evidence IDs alongside the relevant text and keep an authoritative ID-to-source mapping for rendering and validation. Citation identity should survive reranking and formatting; retrieval rank is not a page number.

Why the alternatives fail

A. Ranking positions change by query and have no necessary relationship to document pagination.

B. The model cannot reliably recover provenance that the pipeline removed.

C. A shared ID destroys the distinction between independently sourced evidence.

Design takeaway: Provenance is part of the retrieval-to-generation contract, not decorative output text.

Official references: [12]

13. Using MMR for a specific failure mode

Answer D: Retrieve a broader candidate pool, then use MMR to balance relevance and diversity for the final k.

MMR selects from a candidate pool with a trade-off between relevance and redundancy. A broader fetch_k can give it alternatives beyond the near-duplicates, while k controls the final context size. Evaluate whether the added diversity actually covers relevant causes; diversity alone is not proof of answer completeness.

Why the alternatives fail

A. Uniformly shifting scores does not change the ranking or its redundancy.

B. Generation cannot responsibly replace missing evidence with invented causes.

C. A smaller candidate pool constrains the choices available to the selection algorithm.

Design takeaway: Use diversity-aware selection when redundancy, rather than raw result count, is the problem.

Official references: [13]

14. Interpreting score direction

Answer A: Lower distance means closer here; the filter retains the weakest of these matches.

Under the stated distance contract, A is closest and C is farthest. A high-score similarity threshold cannot be copied unchanged onto raw distances. Correct the direction and calibrate any acceptance threshold on representative queries; closeness itself is not a calibrated probability that the answer is supported.

Why the alternatives fail

B. That interpretation contradicts the explicitly stated squared-L2 metric.

C. Distances are not a probability distribution over documents.

D. Wrapping a search interface does not by itself guarantee a probabilistic score conversion.

Design takeaway: Inspect the integration's score semantics before setting thresholds.

Official references: [14]

15. Chunking structured evidence

Answer B: Preserve table/header relationships in chunks and retain document provenance.

The failure is loss of structural meaning, so segmentation should preserve the context that gives a cell its interpretation. Keep table headers or repeat the relevant header context with each coherent table segment. Then evaluate answerable examples involving units and columns. More overlap can help boundaries but is not a substitute for table-aware representation.

Why the alternatives fail

A. Excessive overlap increases redundancy and cost without reliably reconstructing table relationships.

C. Headers carry the very semantics missing from the retrieved cells.

D. Inferring units replaces authoritative document structure with an avoidable guess.

Design takeaway: Choose chunk boundaries that preserve the evidence needed to interpret each fragment.

Official references: [15]

16. Understanding the reranker recall ceiling

Answer C: Improve candidate retrieval so the required passage can enter the reranker's input set.

A reranker can reorder or filter the candidates it receives; it cannot select a passage that was never retrieved. Investigate query formulation, indexing coverage, retrieval methods, and candidate depth. Increasing final k may improve coverage among existing candidates but cannot fix this particular missing-passage case.

Why the alternatives fail

A. Temperature does not provide evidence absent from the context.

B. The missing passage is not in the pool, so selecting more of that same pool cannot include it.

D. Presentation changes do not alter candidate coverage.

Design takeaway: Measure candidate recall separately from ranking quality.

Official references: [16]

17. Designing discriminative tool schemas

Answer C: Define distinct tool purposes, temporal applicability, and validated date/identifier fields.

The model uses tool names, descriptions, and argument schemas to decide what to request. Distinguish current versus historical scope and state the relevant date semantics. Validate arguments at execution time as well. This improves selection and catches malformed requests, while leaving actual authorization and data correctness to the application.

Why the alternatives fail

A. Repeating an ambiguous instruction does not resolve its missing information.

B. Opaque labels remove useful semantic cues rather than clarifying the distinction.

D. More indistinguishable tools expand the ambiguity.

Design takeaway: Tool descriptions and schemas are operational interfaces that need precise semantics.

Official references: [17]

18. Blocking work inside an async handler

Answer B: Await chain.ainvoke and audit custom stages for blocking calls; offload unavoidable synchronous work.

Calling a blocking synchronous method directly inside an async handler can occupy the event-loop thread. Awaiting the async interface allows other tasks to progress while supported I/O waits. Custom code still needs inspection: labeling a function async does not make a synchronous library call nonblocking. Use an appropriate executor or async client where necessary.

Why the alternatives fail

A. Longer responses can increase the blocking interval rather than resolving it.

C. An unawaited coroutine does not deliver the required result and may never run.

D. Busy waiting consumes execution capacity and does not yield cooperative progress.

Design takeaway: Async behavior must hold across the whole request path, including custom stages.

Official references: [10]

19. Text parsing versus a typed contract

Answer D: Use structured output or JSON parsing plus schema validation before calling the service.

StrOutputParser returns text, so the service has not yet received its required object contract. Parse and validate the structure or obtain a validated structured response directly. In Python, bool(nonempty_string) is True even when that string contains the word false; lexical presence is also not the decision value.

Why the alternatives fail

A. The field name can appear with either True or False and therefore cannot determine the decision.

B. A serialized representation is different from the parsed object it describes.

C. That conversion tests string emptiness rather than the JSON Boolean value.

Design takeaway: Do not confuse serialized text with a validated application object.

Official references: [18]

20. Choosing deterministic retrieval placement

Answer A: Run a fixed retrieval-then-generation chain, with an explicit insufficient-evidence path.

The mandatory retrieval requirement is naturally enforced by application control flow. A two-step chain ensures evidence is gathered before generation, and an explicit insufficient-evidence path prevents unsupported answers when retrieval fails. Agentic retrieval is useful when dynamic search planning is needed, but discretionary retrieval is not the requirement here.

Why the alternatives fail

B. An optional model decision does not enforce a mandatory workflow step.

C. Post-hoc retrieval risks attaching citations to claims generated without that evidence.

D. Calling an unrelated tool does not satisfy consultation of the approved knowledge base.

Design takeaway: Choose orchestration complexity from the requirement, not from the appeal of agents.

Official references: [19]

LangChain-M-Set1

Official sources | Documentation checked October 28, 2026

These original scenarios apply documented APIs and engineering reasoning to the stated constraints. References support the concepts; they do not prescribe every architectural conclusion. Standard imports and setup are omitted from illustrative snippets.

[1] RunnablePassthrough
https://reference.langchain.com/python/langchain-core/runnables/passthrough/RunnablePassthrough

[2] RunnableAssign
https://reference.langchain.com/python/langchain-core/runnables/passthrough/RunnableAssign

[3] RunnableParallel
https://reference.langchain.com/python/langchain-core/runnables/base/RunnableParallel

[4] RunnableLambda and streaming
https://reference.langchain.com/python/langchain-core/runnables/base/RunnableLambda

[5] Completion-order batch results
https://reference.langchain.com/python/langchain-core/runnables/base/Runnable/batch_as_completed

[6] Chat model interface
https://docs.langchain.com/oss/python/langchain/models

[7] Pydantic models and validation
https://docs.pydantic.dev/latest/concepts/models/

[8] Agent structured output
https://docs.langchain.com/oss/python/langchain/structured-output

[9] MessagesPlaceholder
https://reference.langchain.com/python/langchain-core/prompts/chat/MessagesPlaceholder

[10] Runnable composition
https://reference.langchain.com/python/langchain-core/runnables/base/Runnable

[11] Embedding integrations and interface
https://docs.langchain.com/oss/python/integrations/embeddings

[12] Document content and metadata
https://reference.langchain.com/python/langchain-core/documents/base/Document

LangChain-M-Set1

Official sources | Documentation checked October 28, 2026

These original scenarios apply documented APIs and engineering reasoning to the stated constraints. References support the concepts; they do not prescribe every architectural conclusion. Standard imports and setup are omitted from illustrative snippets.

[13] VectorStore search interface
https://github.com/langchain-ai/langchain/blob/master/libs/core/langchain_core/vectorstores/base.py

[14] FAISS score implementation
https://github.com/langchain-ai/langchain-community/blob/main/libs/community/langchain_community/vectorstores/faiss.py

[15] Text splitter integrations
https://docs.langchain.com/oss/python/integrations/splitters

[16] Reranker integration
https://docs.langchain.com/oss/python/integrations/retrievers/cohere-reranker

[17] Tool schemas and execution context
https://docs.langchain.com/oss/python/langchain/tools

[18] String output parser
https://reference.langchain.com/python/langchain-core/output_parsers/string/StrOutputParser

[19] Agent execution and tool loop
https://docs.langchain.com/oss/python/langchain/agents